

Hard-Coding ingest.py in Python

A line-by-line tutorial: what every part does, why it exists, and how to write it yourself from scratch.

Project: logistics-rag-agent

File: ingest.py

Pipeline: PDF files -> text chunks -> embeddings -> Chroma vector DB

This tutorial teaches you to rebuild ingest.py by hand. You will not just copy-paste: you will understand imports, paths, idempotent skip logic, chunking, embeddings, and persistence.

1. Big picture: what ingest.py is for

A RAG (Retrieval-Augmented Generation) app answers questions using YOUR documents. Before the chat agent can search those documents, you must load them into a searchable index. That one-time (and re-runnable) job is called ingestion.

The four stages

- LOAD: read PDF bytes and turn each page into text + metadata
- SPLIT: cut long pages into overlapping chunks (better search units)
- EMBED: turn each chunk into a vector of numbers (semantic meaning)
- STORE: save vectors + text + metadata into Chroma on disk

WHY

Search later compares the question's embedding to stored chunk embeddings. If you skip ingest, the agent has nothing to retrieve.

What 'hard code' means here

Hard-coding means writing the script yourself with explicit constants and clear steps -- not hiding the flow inside a magic framework CLI. Your values (folder paths, model name, chunk size) are written in the file so you can see and change them.

2. Prerequisites before you type code

- Python 3.10+ and a virtualenv (this project uses rag-env/)
- Ollama running locally with model: nomic-embed-text
- Packages: langchain-chroma, langchain-community, langchain-ollama, langchain-text-splitters, pypdf
- A data/ folder next to ingest.py containing one or more .pdf files

WHY

OllamaEmbeddings talks to a local Ollama server. If Ollama is not running or the embed model is missing, add_documents will fail when embeddings are computed.

```
ollama pull nomic-embed-text
pip install langchain-chroma langchain-community langchain-ollama \
            langchain-text-splitters pypdf
```

3. Build the file step by step

Step 1: Create the file and write the docstring

```
"""Ingest PDFs from data/ into Chroma. Safe to re-run -- skips already-stored files."""
```

A module docstring is the first string in the file. It documents purpose for humans and for tools (help, IDEs). The important promise here: safe to re-run.

WHY

Ingestion is expensive (CPU + time). Re-running should not duplicate every PDF. The docstring states that contract so future-you does not 'fix' it by always wiping the DB.

Step 2: Import only what you need

```
from pathlib import Path

from langchain_chroma import Chroma
from langchain_community.document_loaders import PyPDFLoader
from langchain_ollama import OllamaEmbeddings
from langchain_text_splitters import RecursiveCharacterTextSplitter
```

What each import is

- Path: modern filesystem paths (join folders, glob PDFs, resolve absolute paths)
- Chroma: LangChain wrapper around the Chroma vector database
- PyPDFLoader: reads a PDF into a list of Document objects (one per page)
- OllamaEmbeddings: calls your local Ollama model to embed text
- RecursiveCharacterTextSplitter: splits text on natural boundaries (paragraphs, sentences, words)

WHY

We import from package-specific modules (langchain_chroma, langchain_ollama) instead of old catch-all community paths when possible -- clearer dependencies, fewer deprecation warnings.

Step 3: Hard-code configuration as named constants

```
DATA_DIR = Path(__file__).parent / "data"
CHROMA_DIR = Path(__file__).parent / "chroma_db"
COLLECTION = "logistics_docs"
EMBED_MODEL = "nomic-embed-text"

CHUNK_SIZE = 680
CHUNK_OVERLAP = 100
```

Line-by-line meaning

- Path(__file__).parent -- directory containing ingest.py, no matter where you run the command from
- DATA_DIR -- where source PDFs live (project/data/)
- CHROMA_DIR -- where Chroma writes its on-disk database (project/chroma_db/)
- COLLECTION -- named bucket inside Chroma; like a table name for vectors
- EMBED_MODEL -- must match what query/RAG code uses later or search breaks
- CHUNK_SIZE = 680 -- aim for ~680 characters per chunk (not words)
- CHUNK_OVERLAP = 100 -- last 100 chars of a chunk repeat at the start of the next

WHY

Constants at the top are the 'control panel'. Change chunk size here once instead of hunting through functions. Using `Path(__file__).parent` keeps paths stable when you run: `python ingest.py` from another cwd.

WHY

Overlap exists so a sentence split across a boundary still appears fully in at least one chunk. Without overlap, retrieval can miss answers that straddled the cut.

Step 4: Open (or create) the vector store

```
def get_vectorstore() -> Chroma:
    return Chroma(
        persist_directory=str(CHROMA_DIR),
        embedding_function=OllamaEmbeddings(model=EMBED_MODEL),
        collection_name=COLLECTION,
    )
```

This function does not ingest anything. It returns a live Chroma client pointed at your folder. If chroma_db/ does not exist yet, Chroma creates it when you add documents.

Keyword arguments (library-fixed names)

- persist_directory -- string path on disk; str(...) because Chroma expects str, not Path
- embedding_function -- object that turns text into vectors; must stay consistent
- collection_name -- which collection inside that directory to use

WHY

You do not invent parameter names like persist_directory. Those are defined by LangChain's Chroma.__init__. You only choose the VALUES (your path, your model, your collection string).

WHY

Wrapping this in get_vectorstore() lets ingest.py and rag.py share the same open logic. One place to fix if the API changes.

Step 5: Discover which PDFs are already stored

```
def already_ingested_sources(vectorstore: Chroma) -> set[str]:
    """Absolute source paths already present in Chroma."""
    data = vectorstore.get(include=["metadatas"])
    return {
        meta["source"]
        for meta in (data.get("metadatas") or [])
        if meta and meta.get("source")
    }
```

What .get() returns (shape)

```
{
  "ids": ["id1", "id2", ...],
  "metadatas": [
    {"source": "/abs/path/a.pdf", "filename": "a.pdf", "page": 0},
    {"source": "/abs/path/a.pdf", "filename": "a.pdf", "page": 1},
    ...
  ]
}
```

include=["metadatas"] asks Chroma for metadata only -- not the full text or embeddings. That is cheaper when you only need to know which files exist.

The set comprehension, piece by piece

- data.get("metadatas") or [] -- if metadatas is missing/None, use empty list (no crash)
- for meta in ... -- each chunk has its own metadata dict
- if meta and meta.get('source') -- skip empty/None dicts and dicts without source
- meta["source"] -- the path string we stored at ingest time
- outer { ... } -- a Python set: unique paths only (one PDF -> many chunks -> one path)

WHY

Idempotency: before embedding a PDF again, check if its source path is already in the DB. A set makes membership tests (path in known) $O(1)$ and automatic de-duplication.

Step 6: Write main(): find PDFs

```
def main() -> None:
    pdf_paths = sorted(DATA_DIR.glob("**/*.pdf"))
    if not pdf_paths:
        print(f"No PDFs found in {DATA_DIR}")
        return

    print(f"Found {len(pdf_paths)} PDF(s) in {DATA_DIR}")
    for i, path in enumerate(pdf_paths, start=1):
        print(f"  {i}. {path.name}")
```

- glob("**/*.pdf") -- recursive search for every PDF under data/
- sorted(...) -- stable order so logs look the same each run
- Early return if empty -- fail loudly with a clear message instead of silently doing nothing useful
- enumerate(..., start=1) -- human-friendly numbering in the console

WHY

Printing the inventory first helps debugging: if a PDF is missing from the list, the problem is the folder/filename, not Chroma or Ollama.

Step 7: Filter to only NEW PDFs

```
vectorstore = get_vectorstore()
known = already_ingested_sources(vectorstore)

new_paths = [
    p
    for p in pdf_paths
    if str(p.resolve()) not in known and str(p) not in known
]

if not new_paths:
    print("Nothing new to ingest -- all PDFs already in Chroma.")
    return
```

p.resolve() turns a relative path into an absolute path (symlink-aware). We check BOTH resolve() and str(p) because older runs might have stored either form.

WHY

Defensive matching avoids re-embedding the same file just because one run saved 'data/a.pdf' and another saved '/Users/.../data/a.pdf'.

Step 8: Create the text splitter once

```
splitter = RecursiveCharacterTextSplitter(
    chunk_size=CHUNK_SIZE,
    chunk_overlap=CHUNK_OVERLAP,
)
```

RecursiveCharacterTextSplitter tries to split on a list of separators in order (paragraph breaks, newlines, spaces, then characters). That keeps chunks more readable than a naive every-N-characters cut.

WHY

Create the splitter once outside the PDF loop. The object is reusable; recreating it per file wastes nothing critical but is pointless noise.

Step 9: Load, normalize metadata, split, embed+store

```
total_chunks = 0
for path in new_paths:
    pages = PyPDFLoader(str(path)).load()
    # Normalize source so re-runs match reliably
    for page in pages:
        page.metadata["source"] = str(path.resolve())
        page.metadata["filename"] = path.name

    chunks = splitter.split_documents(pages)
    if not chunks:
        print(f"  skip (empty): {path.name}")
        continue

    vectorstore.add_documents(chunks)
    total_chunks += len(chunks)
    print(f"  + {path.name}: {len(pages)} pages -> {len(chunks)} chunks")

print(f"\nDone. Added {total_chunks} chunks -> {CHROMA_DIR}")
```

Inside the loop -- exact sequence

- PyPDFLoader(str(path)).load() -- open PDF; return List[Document], usually one Document per page
- page.metadata['source'] = str(path.resolve()) -- overwrite with absolute path (skip logic depends on this)
- page.metadata['filename'] = path.name -- short name for humans/debugging
- splitter.split_documents(pages) -- pages become smaller Document chunks; metadata is copied to each chunk
- if not chunks: continue -- empty/scanned-image PDFs with no extractable text should not call embed
- vectorstore.add_documents(chunks) -- embed each chunk via Ollama, then write to Chroma disk
- print progress -- ops-friendly visibility: pages -> chunks per file

WHY

Metadata normalization BEFORE split is important: split_documents copies page metadata onto each chunk. If you set source after splitting, you must loop chunks instead -- more code, same result. Do it once on pages.

WHY

add_documents is the heavy step. It calls the embedding model for every chunk, then persists. That is why skipping known sources matters.

Step 10: Make the file executable as a script

```
if __name__ == "__main__":
    main()
```

When you run python ingest.py, Python sets __name__ to '__main__' and this block runs. When another file does import ingest, main() does NOT auto-run -- only functions are available.

WHY

This pattern lets you unit-test helpers (get_vectorstore, already_ingested_sources) without accidentally starting a full ingest on import.

4. Full hard-coded file (reference)

After following the steps, your ingest.py should look like this:

```
"""Ingest PDFs from data/ into Chroma. Safe to re-run -- skips already-stored files."""

from pathlib import Path

from langchain_chroma import Chroma
from langchain_community.document_loaders import PyPDFLoader
from langchain_ollama import OllamaEmbeddings
from langchain_text_splitters import RecursiveCharacterTextSplitter

DATA_DIR = Path(__file__).parent / "data"
CHROMA_DIR = Path(__file__).parent / "chroma_db"
COLLECTION = "logistics_docs"
EMBED_MODEL = "nomic-embed-text"

CHUNK_SIZE = 680
CHUNK_OVERLAP = 100

def get_vectorstore() -> Chroma:
    return Chroma(
        persist_directory=str(CHROMA_DIR),
        embedding_function=OllamaEmbeddings(model=EMBED_MODEL),
        collection_name=COLLECTION,
    )

def already_ingested_sources(vectorstore: Chroma) -> set[str]:
    """Absolute source paths already present in Chroma."""
    data = vectorstore.get(include=["metadatas"])
    return {
        meta["source"]
        for meta in (data.get("metadatas") or [])
        if meta and meta.get("source")
    }

def main() -> None:
    pdf_paths = sorted(DATA_DIR.glob("**/*.pdf"))
    if not pdf_paths:
        print(f"No PDFs found in {DATA_DIR}")
        return

    print(f"Found {len(pdf_paths)} PDF(s) in {DATA_DIR}")
    for i, path in enumerate(pdf_paths, start=1):
        print(f" {i}. {path.name}")

    vectorstore = get_vectorstore()
    known = already_ingested_sources(vectorstore)

    new_paths = [
        p for p in pdf_paths
        if str(p.resolve()) not in known and str(p) not in known
    ]

    if not new_paths:
        print("Nothing new to ingest -- all PDFs already in Chroma.")
```

```
        return

    print(f"\nIngesting {len(new_paths)} new PDF(s); "
          f"skipping {len(pdf_paths) - len(new_paths)} already stored.")

    splitter = RecursiveCharacterTextSplitter(
        chunk_size=CHUNK_SIZE,
        chunk_overlap=CHUNK_OVERLAP,
    )

    total_chunks = 0
    for path in new_paths:
        pages = PyPDFLoader(str(path)).load()
        for page in pages:
            page.metadata["source"] = str(path.resolve())
            page.metadata["filename"] = path.name

        chunks = splitter.split_documents(pages)
        if not chunks:
            print(f"  skip (empty): {path.name}")
            continue

        vectorstore.add_documents(chunks)
        total_chunks += len(chunks)
        print(f"  + {path.name}: {len(pages)} pages -> {len(chunks)} chunks")

    print(f"\nDone. Added {total_chunks} chunks -> {CHROMA_DIR}")

if __name__ == "__main__":
    main()
```

5. Data flow (mental model)

```
data/*.pdf
|
| PyPDFLoader.load()
v
List[Document]          # one Document per page; .page_content + .metadata
|
| metadata['source'] = absolute path
| RecursiveCharacterTextSplitter.split_documents()
v
List[Document]          # many smaller chunks; metadata copied
|
| OllamaEmbeddings (nomic-embed-text)
| Chroma.add_documents()
v
chroma_db/              # vectors + text + metadata on disk
|
| later: rag.py / agent similarity_search(question)
v
Top-k relevant chunks for the LLM prompt
```

WHY

Ingest writes the library. Query/RAG only reads it. Keep EMBED_MODEL, COLLECTION, and CHROMA_DIR identical on both sides or retrieval returns garbage/empty.

6. Glossary of every important term

- Document -- LangChain object with page_content (str) and metadata (dict)
- Chunk -- a Document piece after splitting; the unit you embed and retrieve
- Embedding -- list/vector of floats representing semantic meaning of text
- Vector store -- database specialized for nearest-neighbor search over embeddings
- Collection -- named group of vectors inside one Chroma persist directory
- Metadata -- side information (source path, filename, page number) stored with each chunk
- Idempotent -- running the same operation twice does not create duplicate work/data
- Persist directory -- folder where Chroma saves its database files

7. Practice checklist (prove you understand)

- Delete chroma_db/, run python ingest.py, confirm chunks are added
- Run python ingest.py again -- expect 'Nothing new to ingest'
- Add a new PDF under data/, re-run -- only the new file should embed
- Temporarily change EMBED_MODEL in rag only -- observe broken/empty search (then fix)
- Change CHUNK_SIZE to 200, wipe DB, re-ingest -- note more chunks, different retrieval
- Print already_ingested_sources(get_vectorstore()) in a REPL and read the paths

8. Common failure modes

- No PDFs found -- wrong DATA_DIR or empty data/; check Path(__file__).parent
- Ollama connection error -- start Ollama; pull nomic-embed-text
- Always re-ingesting -- source metadata not normalized to resolve(); skip check fails

- Empty chunks -- PDF is image-only; need OCR (out of scope for this script)
- Import errors -- activate rag-env and install the langchain packages listed above

You are done when you can explain every line out loud without looking.

If you can teach Steps 4-9 to someone else -- vectorstore, skip set, split, metadata, add_documents -- you truly hard-coded ingest.py, not just pasted it.